

HDOC – Experiment 12

Question

Write a menu-driven C program using switch-case to input a positive integer and perform one of the following operations:

1. Find the sum and product of the first and last digits.
2. Input a digit k and count how many digits of the number are:
 - lesser than k
 - equal to k
 - greater than k

❖ Understanding problem

Input

- $num \rightarrow$ a positive integer
- $choice \rightarrow$ menu choice (1 or 2)
- $k \rightarrow$ a digit from 0 to 9, required only for choice 2

Output

For **Choice 1:**

- First digit
- Last digit
- Sum of first and last digits
- Product of first and last digits

For **Choice 2:**

- Number of digits lesser than k
- Number of digits equal to k

- Number of digits greater than k

❖ Algorithm

1. Start.
2. Input a positive integer `num`.
3. Check whether `num` is positive.
4. If `num <= 0`, display "Please enter a positive integer" and terminate.
5. Display the menu:
 - 5.1 Sum and product of first and last digits
 - 5.2 Count digits lesser than, equal to, and greater than k
6. Input choice.
7. Use `switch(choice)`.
8. If `choice = 1`:
 - 8.1 Find the last digit using `last = num % 10`.
 - 8.2 Copy `num` into `temp`.
 - 8.3 Repeatedly divide `temp` by 10 while `temp >= 10`.
 - 8.4 The remaining value of `temp` is the first digit.
 - 8.5 Calculate `sum = first + last`.
 - 8.6 Calculate `product = first * last`.
 - 8.7 Display the first digit, last digit, sum, and product.
9. If `choice = 2`:
 - 9.1 Input digit k.
 - 9.2 Check whether k is between 0 and 9.
 - 9.3 Initialize `lesser = 0`, `equal = 0`, and `greater = 0`.
 - 9.4 Copy `num` into `temp`.
 - 9.5 Repeat while `temp > 0`:
 - 9.5.1 Find the last digit using `digit = temp % 10`.
 - 9.5.2 If `digit < k`, increment `lesser`.
 - 9.5.3 Else if `digit == k`, increment `equal`.
 - 9.5.4 Else, increment `greater`.
 - 9.5.5 Remove the last digit using `temp = temp / 10`.
 - 9.6 Display `lesser`, `equal`, and `greater`.
10. For any other choice, display "Invalid choice".
11. Stop.

❖ Test case table

TEST CASE	INPUT	EXPECTED OUTPUT	ACTUAL OUTPUT	RESULT
1	num=12345, choice=1	First=1, Last=5, Sum=6, Product=5	SAME	PASS
2	num=9082, choice=1	First=9, Last=2, Sum=11, Product=18	SAME	PASS
3	num=7, choice=1	First=7, Last=7, Sum=14, Product=49	SAME	PASS
4	num=12345, choice=2, k=3	Lesser=2, Equal=1, Greater=2	SAME	PASS
5	num=33321, choice=2, k=3	Lesser=2, Equal=3, Greater=0	SAME	PASS
6	num=78999, choice=2, k=9	Lesser=2, Equal=3, Greater=0	SAME	PASS
7	num=12345, choice=3	Invalid choice	SAME	PASS
8	num=-123, choice=1	Please enter a positive integer	SAME	PASS

❖ Program in C

```
#include <stdio.h>

int main()
{
    int num, temp, choice;

    int first, last, sum, product;
```

```
int k, digit;

int lesser = 0, equal = 0, greater = 0;


// Input positive integer

printf("Enter a positive integer: ");

scanf("%d", &num);


// Validate the number

if (num <= 0)

{

    printf("Please enter a positive integer.\n");

    return 0;

}


// Display menu

printf("\nMENU\n");

printf("1. Find sum and product of first and last digits\n");

printf("2. Count digits lesser than, equal to, and greater than k\n");

printf("Enter your choice: ");

scanf("%d", &choice);


switch (choice)

{

    case 1:

        // Find last digit

        last = num % 10;


        // Find first digit
```

```
temp = num;
while (temp >= 10)
{
    temp = temp / 10;
}
first = temp;

// Calculate sum and product
sum = first + last;
product = first * last;

printf("\nFirst digit = %d\n", first);
printf("Last digit = %d\n", last);
printf("Sum = %d\n", sum);
printf("Product = %d\n", product);
break;
```

case 2:

```
printf("Enter digit k (0-9): ");
scanf("%d", &k);

// Validate k
if (k < 0 || k > 9)
{
    printf("Invalid digit k.\n");
    break;
}
```

```
temp = num;

while (temp > 0)
{
    // Extract last digit
    digit = temp % 10;

    if (digit < k)
    {
        lesser++;
    }
    else if (digit == k)
    {
        equal++;
    }
    else
    {
        greater++;
    }

    // Remove last digit
    temp = temp / 10;
}

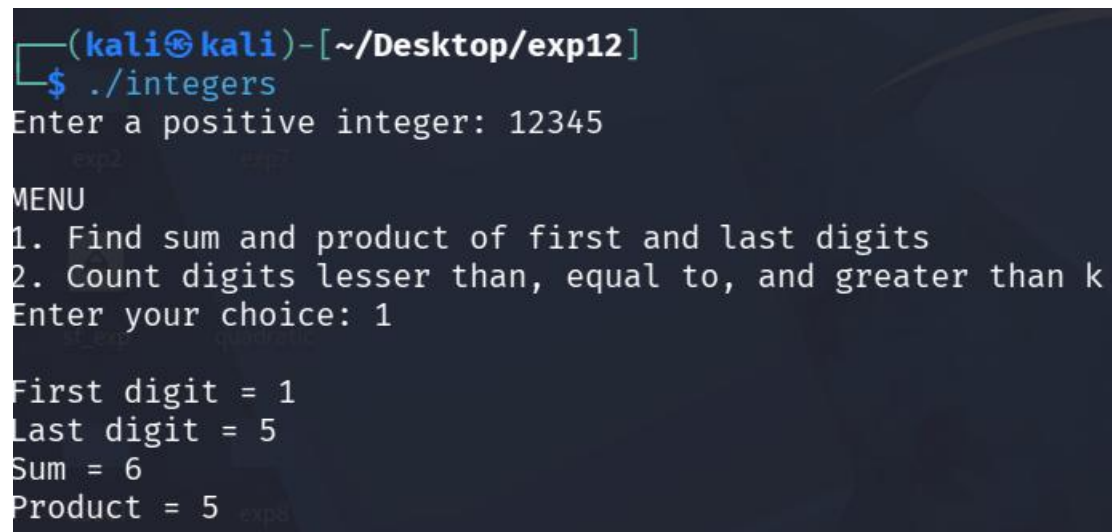
printf("\nCount_lesser = %d\n", lesser);
printf("Count_equal = %d\n", equal);
printf("Count_greater = %d\n", greater);
break;
```

```
        default:
            printf("Invalid choice.\n");
        }

    return 0;
}
```

❖ Compile, Execute and Evaluate

Test case 1



```
(kali㉿kali)-[~/Desktop/exp12]
└─$ ./integers
Enter a positive integer: 12345
MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1
First digit = 1
Last digit = 5
Sum = 6
Product = 5
```

Test case 2

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 9082

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1

First digit = 9
Last digit = 2
Sum = 11
Product = 18
```

Test case 3

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 7

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1

First digit = 7
Last digit = 7
Sum = 14
Product = 49
```

Test case 4


```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 12345

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 2
Enter digit k (0-9): 3

Count_lesser = 2
Count_equal = 1
Count_greater = 2
```

Test case 5

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 78999

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 2
Enter digit k (0-9): 9

Count_lesser = 2
Count_equal = 3
Count_greater = 0
```

Test case 6

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 12345

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 3
Invalid choice.
```

Test case 7

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: -123
Please enter a positive integer.
```

❖ Compilation and execution screenshot

```
(kali㉿kali)-[~/Desktop/exp12]
$ vi integers.c

(kali㉿kali)-[~/Desktop/exp12]
$ gcc integers.c -o integers -lm

(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 12345

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1

First digit = 1
Last digit = 5
Sum = 6
Product = 5

(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 9082

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1

First digit = 9
Last digit = 2
Sum = 11
Product = 18
```

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 7

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 1

First digit = 7
Last digit = 7
Sum = 14
Product = 49

(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 12345

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 2
Enter digit k (0-9): 3

Count_lesser = 2
Count_equal = 1
Count_greater = 2

(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
Enter a positive integer: 78999

MENU
1. Find sum and product of first and last digits
2. Count digits lesser than, equal to, and greater than k
Enter your choice: 2
Enter digit k (0-9): 9

Count_lesser = 2
```

```
Count_lesser = 2
Count_equal = 3
Count_greater = 0
```

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
```

Enter a positive integer: 12345

MENU

1. Find sum and product of first and last digits
 2. Count digits lesser than, equal to, and greater than k
- Enter your choice: 3
Invalid choice.

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
```

Enter a positive integer: 123

MENU

1. Find sum and product of first and last digits
 2. Count digits lesser than, equal to, and greater than k
- Enter your choice: 1

First digit = 1
Last digit = 3
Sum = 4
Product = 3

```
(kali㉿kali)-[~/Desktop/exp12]
$ ./integers
```

Enter a positive integer: -123
Please enter a positive integer.